

A Colorful Journey Through the Recursive Stack

Understanding `fun(3)` step by step

Big idea: recursion first **goes down** by creating stack frames, then **comes back up** by finishing them.

For this program, nothing is printed while calls are going down. The output appears only while the stack unwinds: **1 2 3**

The Code

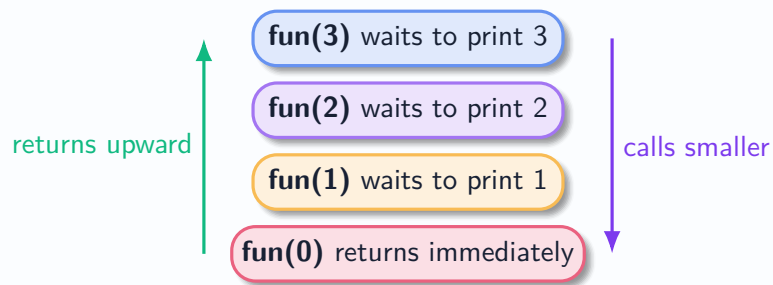
```
1 def fun(n):
2     if n == 0:
3         return
4     fun(n-1)
5     print(n, end=' ')
6
7 fun(3)
```

What Each Line Means

Line	Role in the recursion
<code>def fun(n):</code>	Creates a function. Each call receives its own private value of <code>n</code> .
<code>if n == 0: return</code>	The base case : it stops the recursion so calls do not continue forever.
<code>fun(n-1)</code>	The recursive call : pause the current call and ask a smaller call to finish first.
<code>print(n, end=' ')</code>	Runs <i>after</i> the smaller call returns, so printing happens during unwinding.

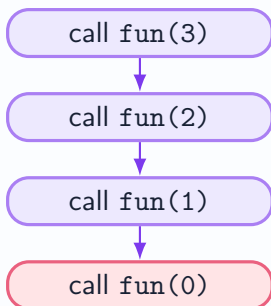
Mental Model: A Stack of Paused Calls

Think of each function call as a card placed on a stack. Python cannot print `n` yet because every call must first wait for `fun(n-1)` to complete.



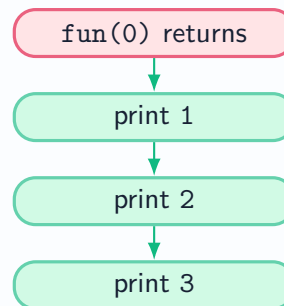
Two Phases of Execution

1. Going down



No printing yet; calls are being stacked.

2. Coming back up



Paused print statements finally execute.

Full Trace

Step	Action	Stack state	Output
1	Start fun(3)	[fun(3)]	—
2	fun(3) calls fun(2)	[fun(3), fun(2)]	—
3	fun(2) calls fun(1)	[fun(3), fun(2), fun(1)]	—
4	fun(1) calls fun(0)	[fun(3), fun(2), fun(1), fun(0)]	—
5	fun(0) hits base case	[fun(3), fun(2), fun(1)]	—
6	Resume fun(1) and print	[fun(3), fun(2)]	1
7	Resume fun(2) and print	[fun(3)]	1 2
8	Resume fun(3) and print	[]	1 2 3

Why the Output Is Not 3 2 1

The print statement is **after** the recursive call. That means every call says: “first solve the smaller problem, then print my number.” Since fun(1) is the first non-base call to finish after fun(0) returns, it prints first.

Golden rule: code *before* the recursive call runs while going down; code *after* the recursive call runs while coming back up.

If we moved `print(n, end=' ')` above `fun(n-1)`, the output would become 3 2 1.

Common Beginner Confusions

Confusion 1

“Does return print something?”

No. return only sends control back to the previous waiting call. The printing happens later at `print(n, end=' ')`.

Confusion 2

“Is there only one `n`?”

No. Each call has its own `n`. So `fun(3)`, `fun(2)`, and `fun(1)` are separate paused frames.

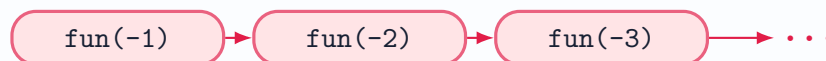
Confusion 3

“Why wait before printing?”

Because Python must finish `fun(n-1)` before it can move to the next line. The current call is paused.

Tiny Edge Cases

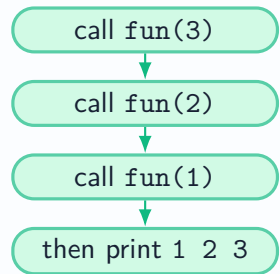
Call	What happens	Output
<code>fun(0)</code>	Immediately hits the base case. No recursive call is made.	Nothing is printed.
<code>fun(1)</code>	Calls <code>fun(0)</code> , returns, then prints	1
<code>fun(3)</code>	Builds three waiting frames, then prints while unwinding.	1 2 3
<code>fun(-1)</code>	This version has no stopping rule for negative numbers, so it keeps calling <code>fun(-2)</code> , <code>fun(-3)</code> , ...	Error eventually: recursion goes too deep.



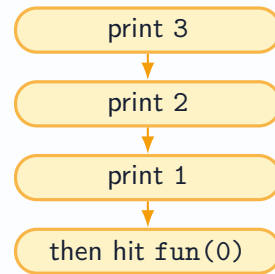
Since `n == 0` is never reached, negative input is an edge case that breaks this beginner version.

What Changes If We Move print?

Print after call



Print before call



Beginner checkpoint: when reading recursive code, put your finger on the recursive call. Anything above it happens on the way **down**; anything below it happens on the way **back up**.

Final Answer

Output: 1 2 3